

Repartitioning Convention: Pebbles, Punched Cards, and the Migration of Schema

Flyxion

Independent Researcher

September 2026

Abstract

A punched card is often described as a binary medium: a position either holds a hole or it does not. This description is correct at the level of physical sensing and almost useless as an account of what the card represents. This essay argues that discreteness and symbolic notation are separable achievements, and that a data record's computability never entails that every computable operation upon it is meaningful. Beginning with a pre-symbolic accounting procedure, pebbles matched one-to-one against sheep passing through a gate, the essay traces a progression in which discrete correspondence becomes progressively enriched: tally marks, positional numerals, the multi-field punched card introduced by IBM in 1928, the header-bearing CSV file, and the explicitly typed database or JSON schema. What changes across this sequence is not the presence of discreteness, which is constant from the first pebble onward, but the location of the convention that assigns meaning to a discrete state. In the pebble bowl, convention resides almost entirely in an embodied procedure. In the punched-card installation, it is distributed across card design, printed headings, plugboard wiring, operating manuals, and trained personnel. In CSV and in typed schema formats, part of that convention migrates into the machine-readable channel itself. At no stage does the artifact fully explain itself. The essay proposes a three-gate notion of admissibility, distinguishing what a machine can mechanically execute upon a field, what a declared type licenses, and what the context of a calculation actually warrants, and argues that the gap between these gates does not close as formats become more sophisticated. It is instead repartitioned: portions of it are absorbed into type systems and validation logic, while the deepest layer, contextual warrant, remains dependent on documentation, institutional practice, and trained judgment at every stage considered here.

1 Introduction

Ask what a punched card stores and the first answer is usually binary: at each of several hundred possible positions, a hole is present or it is not. A standard IBM card of the format introduced in 1928 carries eighty columns and twelve rows, for 960 possible punch positions; the earlier round-hole cards it superseded carried fewer. This is true, and it explains almost nothing about what the card represents. A punched card used in mid-twentieth-century business machines did not encode a single large binary number. It encoded a transaction, understood as a heterogeneous tuple of typed values: a date, a customer code, a town code, a salesperson code, a quantity, a commodity code, a price, a cost, and a freight charge. Each of these fields has its own domain of legitimate values and its own set of operations that make sense to perform upon it. Quantities can be summed. Freight charges can be summed. A state or town code can be grouped but not meaningfully averaged. A customer code can be sorted and counted but not added. The physical substrate, a hole or the

absence of one, is blind to all of this. It supplies presence and absence; it does not supply the categories that make presence and absence significant.

This essay is about where that significance comes from, and about how the location of that significance has shifted historically as recording technologies have changed. The argument proceeds through a sequence of representational systems, each more capable than the last of preserving structure without external support, and each still leaving something essential outside the artifact itself. The sequence begins before any written or spoken symbol exists at all, with a minimal accounting procedure built from loose tokens.

2 Correspondence Without Symbols

Consider a minimal accounting procedure, historically plausible wherever livestock and loose tokens coexist, that requires neither number words nor written numerals. A herder needs to know whether every sheep that left the pen in the morning has returned by evening, without possessing a counting vocabulary. A simple procedure suffices. As each sheep passes through the gate at departure, the herder places one pebble into a bowl. As each sheep returns in the evening, the herder removes one pebble from the bowl and sets it aside.

This procedure establishes a one-to-one correspondence between two collections without requiring a name for the cardinality of either one. It yields three progressively stronger results, and it is worth separating them, since only the first two are available without any numeral system at all. If the bowl is empty once the returning flock has passed through the gate, departures and returns have been exactly matched. If pebbles remain in the bowl, at least one sheep is unaccounted for; this is already known before anyone counts anything. The remaining collection also preserves the magnitude of that deficit, in the sense that it could in principle be matched against some other reference collection, but naming that magnitude, saying there are four missing rather than some unspecified number, requires either further one-to-one comparison against a known reference set or an act of counting that goes beyond the correspondence procedure itself.

Formally, let P denote the current multiset of unmatched departure tokens. Each departing sheep s_i licenses the addition of one token to P . Each returning sheep licenses a state transition that removes one token from P :

$$(P, s'_i) \mapsto (P \setminus \{p\}, s'_i), \quad p \in P.$$

After every departure and every return has been processed exactly once,

$$|P_{\text{final}}| = |S_{\text{departing}}| - |S_{\text{returning}}|.$$

That final proviso, that every departure and every return was registered exactly once, is not a formality. Unary correspondence is not immune to error; a mismatched or double-counted token corrupts the result just as surely as an error in a written ledger would, and the procedure's reliability depends entirely on the fidelity with which it is carried out, not on any property of the pebbles themselves.

It is worth being precise about what this notation is and is not. It is the essay's retrospective mathematical redescription of the procedure, written in a formalism the herder has no need of. The herder's actual practice requires no arithmetic, no numerals, and no spoken count for the first two of the three results above. It requires only a rule: deposit one token per departure, withdraw one token per return, and treat a nonempty remainder as a signal that something has gone wrong. A single pebble does not mean "one" in any abstract or portable sense. It means something closer to

“one departure not yet matched by a return,” and its meaning is entirely relational, holding only with respect to the paired event that licensed its deposit and the paired event that will license its removal.

This is already a fully discrete information system in the relevant sense: it partitions a continuous flow of events, sheep crossing a threshold at various times, into countable, individuated units, each of which is registered by a distinct material change of state. It is digital in the older sense of the word, meaning discrete selection among alternatives, well before it is symbolic in the sense of employing an inscription that stands for something else according to a shared code. No one needs to interpret a pebble as a numeral. No one needs to read anything. The system works by structural correspondence, one physical token per event, rather than by representation, one written mark standing in for a quantity.

The pebble bowl also illustrates, in an unusually clean form, a distinction between a witness and an explanation. The bowl, at the end of the day, can attest that a discrepancy exists and, with further comparison, how large it is; it cannot say which sheep is missing, when it went missing, or why. It preserves a lossy invariant, the unmatched cardinality of two event streams, and little else. It is a witness of discrepancy, not a representation of mechanism, a distinction that recurs later in a more developed form, since much of what changes across the sequence considered in this essay is how much of the underlying mechanism a record preserves alongside its bare witness of an outcome.

Two further features of the pebble system deserve emphasis because they anticipate the punched card discussed below. First, the pebbles themselves carry almost no intrinsic semantic content. An observer who encountered the bowl without knowing the procedure that produced it could not determine whether it tracked sheep, debts, days of labor, or votes cast. The same physical tokens, arranged by the same correspondence procedure, could serve any of these purposes; the substrate is indifferent to the domain it happens to be tracking. Second, the convention that gives the pebbles their meaning, one token per departure, one removal per return, a nonempty remainder signals loss, is stored nowhere in the bowl or the pebbles. It resides entirely in the herder’s practice: an embodied, largely tacit procedure that is neither written down nor, in a preliterate setting, expressible in any external medium at all. The pebble system is, in this sense, the limiting case of a broader pattern this essay will trace: an artifact’s discrete states carry meaning only relative to a convention, and that convention can be stored almost anywhere except in the artifact’s material substrate.

2.1 Tallies and Numerals

The pebble bowl has an important limitation: it cannot be inspected after the fact independently of the ongoing procedure, since removing a pebble is itself part of the matching act, and it stores no durable record once the matching is complete. A tally mark, scratched into wood or bone, addresses this by making the discrete unit portable and inspectable without consuming it. Each mark still corresponds to one event, and the notation is still minimally symbolic: a tally of five strokes conveys “five” only through repetition and grouping, not through an arbitrary sign standing for the concept five.

Positional numerals mark a further step. A written numeral such as 47 no longer represents its referent through one-to-one repetition; it compresses an arbitrarily large cardinality into a fixed number of symbols governed by a place-value code. This is a genuine gain in efficiency and portability, and it is also a genuine increase in the interpretive burden placed on convention, since the symbol 4 in the tens place means something different from the symbol 4 in the ones place, a distinction with no analogue in either the pebble bowl or the tally stick.

Although tallies and numerals can be arranged into tables or ledgers containing multiple categories, spatially grouped, differently oriented, or placed under written headings, their differentiation into distinct fields is supplied entirely by the surrounding document rather than by the inscription itself. A row of tally marks does not know it represents sheep rather than debts; a ledger page's heading tells the reader that. The punched card's distinctive contribution, considered next, was to standardize a fixed, multi-field record whose positions could be addressed repeatedly and mechanically by electromechanical equipment, without a human reader consulting the heading at every pass.

3 The Punched Card as Typed Tuple

The historical artifact motivating this essay is a numeric business card of the kind illustrated in IBM sales and instructional literature from the period surrounding 1928, when IBM introduced the eighty-column, twelve-row, rectangular-hole card format that would remain the industry standard for roughly the next fifty years, superseding an earlier generation of round-hole cards with fewer columns.¹ In this numeric system, a punch in a given row of a given column assigned one of ten decimal values to that column; adjacent columns were grouped into multi-digit fields by a layout fixed in advance. Categories that would today be stored as text, a customer's name, a town, a salesperson's identity, a commodity description, were instead represented as assigned code numbers referring to lookup lists maintained on paper elsewhere in the business, a customer number, a town code, a salesperson number, a commodity code. Alphabetic punching and printing, which allowed later card systems to record text characters directly through combinations of punches within a column, followed in the early 1930s, extending the same positional architecture to a wider character set without changing its underlying logic.

Grouped together, several adjacent columns constitute a field, and several fields together constitute a record representing one business transaction:

$r = (\text{date}, \text{customer code}, \text{town code}, \text{salesperson code}, \text{quantity}, \text{commodity code}, \text{price}, \text{cost}, \text{freight}).$

Each field has its own domain and its own set of admissible operations. A town code and a customer code both occupy numeric character positions and are, at the level of the punching mechanism, structurally identical, yet one denotes a place of business and the other denotes an organizational identity, and no operation that mixes the two produces a meaningful result. A quantity and a price both occupy numeric positions, yet one is a count and the other is a monetary value per unit, and adding them together is as meaningless as adding a customer code to a freight charge, even though the tabulating machine's adder circuit would perform either addition without complaint if wired to do so.

This is the sense in which the card is a digital tuple rather than a binary word. Three layers are present and frequently collapsed in casual description. At the substrate level, each potential punch position is binary: a hole or its absence. At the code level, a punch at a specific row within a specific column designates a decimal digit, so the coding scheme operating over the binary substrate is decimal rather than binary. At the record level, several typed fields are bound together as attributes of a single transaction, making the whole a relational structure rather than an isolated value. Sorting, tabulating, and summing operate over projections of this structure: sort by salesperson code, group by commodity code, sum the quantity field, compare cost against price. None of these operations is intelligible at the substrate level alone; each presupposes that the machine, or more

¹On the introduction and dimensions of the 80-column format, see the sources cited in the bibliography.

precisely the apparatus surrounding the machine, has been configured to treat particular columns as constituting a particular field.

The situated character of a punch is worth stating precisely. A hole at a given coordinate does not, by itself, mean the digit seven. It means the digit seven only relative to the row-column encoding convention; it means the value seven of a particular field only relative to the boundary that delimits that field within the card's width; and it means something economically or organizationally specific, a quantity of seven units, only relative to the schema that assigns quantity to that field in the first place. Formally, the meaning of a punch is a function of several jointly necessary conditions:

$$\text{meaning} = f(\text{presence, row, column, field, schema}).$$

A hole considered apart from this structure, a single perforation reported without its coordinates or its surrounding card, conveys almost nothing.

3.1 Convention Before Code

The crucial fact about the punched card, and the fact most easily lost in a purely electrical description of the medium, is that none of the higher layers in this chain are recorded on the card in a form the tabulating machine can sense. The machine detects holes at coordinates. It does not detect field boundaries, and it certainly does not detect the semantic label "quantity" or "freight." Those facts exist elsewhere: in the printed headings on the card's face, which tell a human operator that a given region corresponds to a given category; in the operating manual, which specifies the intended layout for a given application; in the plugboard wiring, which physically determines which card columns are routed to which counters and accumulators inside the tabulator; and in the trained habits of the clerks who fill out and process the cards.

This produces a layered chain of translation from an economic event to a machine operation:

business distinction $\longrightarrow$ form field $\longrightarrow$ operator action $\longrightarrow$ punched coordinate $\longrightarrow$ machine operation.

Convention precedes code at every link in this chain: the category that a field represents is fixed, by agreement among the humans involved, before any code is punched to instantiate it, and remains fixed by that same agreement afterward, since the machine has no means of discovering or verifying it. Knowledge relevant to the transaction is partitioned across the chain, and no single participant, human or mechanical, needs to hold all of it. A clerk filling out an order form knows what quantity of goods was sold and where the customer is located; the clerk does not need to know at which row-column coordinate that quantity will eventually be perforated. The keypunch operator translates the clerk's written entry into a sequence of punches according to a fixed layout; the operator need not understand freight accounting to perform this translation correctly. The plugboard wiring determines which columns feed which counters; the engineer who wires the board need not know what commodity is being sold, only which columns the application's layout has assigned to the quantity field. The tabulator itself knows only electrical contacts and accumulator states. The manager who reads the resulting printed totals interprets them as meaningful business aggregates without needing to know anything about hole positions or wiring diagrams at all.

This partitioning explains a genuine and often overstated claim made in period marketing material for these machines: their supposed universal applicability across business domains. The same physical equipment, sensing holes and routing electrical contacts, could be repurposed to serve payroll, inventory, insurance claims, or scientific data collection, because the machine's operations, sorting, counting, grouping by category, summing a designated column, are formal and domain-general. What changes from one application to another is entirely external to the machine: a new

card layout, a new set of printed headings, a rewired plugboard, and personnel retrained to the new schema. The machine did not need to be taught what freight or insurance premiums are. It needed only a stable positional convention over which its existing algebra of counting and summing remained applicable. The generality of the tabulating installation is therefore a generality of formal operation, not a generality of understanding, and it is purchased by externalizing all domain-specific meaning into forms, wiring, and trained human judgment that surround the machine rather than residing within it.

3.2 A Three-Gate Criterion for Admissibility

The distinction between what a machine can be wired to compute, what a declared type permits, and what a domain actually warrants deserves a formal statement, since it generalizes well beyond punched cards and since a two-way distinction understates the structure involved. Define an operation O upon a field x , evaluated in a context C , as admissible when three conditions jointly hold: the operation is executable over the machine representation $\rho(x)$ of the field; the operation is licensed by the type $\tau(x)$ that the domain or a schema assigns to the field; and the operation is contextually warranted given the purpose of the calculation.

$$\text{Admissible}(O, x, C) = \text{Executable}(O, \rho(x)) \wedge \text{TypeLicensed}(O, \tau(x)) \wedge \text{ContextuallyWarranted}(O, x, C).$$

These three gates are jointly necessary and can fail independently. Averaging two prices is executable, since both are numeric, and it is type-licensed if the schema declares both fields to be monetary values of the same kind, yet it may still be contextually unwarranted: if the two prices are denominated in different currencies without a recorded exchange rate, or refer to incomparable goods, or ought to be weighted by the quantities sold rather than averaged unweighted, the calculation produces a well-formed number that misrepresents the situation it purports to summarize. The third gate is what makes the criterion do real work, since it is precisely the gate that no format considered in this essay, from the punched card through the most elaborately typed modern schema, succeeds in mechanizing.

The punched-card installation enforces, in effect, only the first gate, and even that only through the plugboard wiring rather than through any check performed by the card itself. A plugboard can route the customer code column to an adder just as readily as it can route the quantity column, since both are numeric character positions from the machine's point of view. The resulting sum is executable; the machine will produce a number. Whether it is type-licensed or contextually warranted is a question the hardware has no means of asking. Enforcing the second and third gates falls entirely to the organization: to clerks who know not to route identifier columns to summation counters, to managers who would notice a nonsensical total, and to the institutional conventions that established which fields may be added, which may only be grouped, and which admit no arithmetic operation whatsoever.

4 The Partial Migration: CSV and the Header Row

A comma-separated values file, or CSV, can represent approximately the same tuple as a punched card while locating part of its schema differently. Consider a record for a single transaction:

town_code,salesperson_code,quantity,price,freight
114,SM7,12,4.75,0.30

The header row associates each column position with a name, and this association is expressed in the same character encoding, and travels through the same computational channel, as the data itself.

A program reading this file can recover that the third field is called quantity without consulting any external document, printed form, or wiring diagram. It can select or reorder fields by their declared names rather than by fixed positional offsets alone, at least when the software reading the file has been written to do so.

This is a genuine, if partial and fragile, internalization of what was previously external. On the punched card, the fact that certain columns constitute the town field exists only in the card's printed layout, the operating manual, and the plugboard wiring; the sensing mechanism itself never perceives that fact. In the CSV file, the analogous fact, that a given column is called `town_code`, is present in the file itself, in the same medium and the same channel as the data it describes, which makes the association more portable, inspectable, and reproducible than it was on the card.

The strength of this claim should not be overstated. The CSV format specification, RFC 4180, treats the header line as optional rather than mandatory, and a great deal of software in practice reads CSV files positionally, by column index, whether or not a header is present, precisely reproducing the punched card's rigidity at the software layer. A header can also be missing, duplicated, silently reordered relative to the data rows by an intervening editing step, or interpreted under an incompatible dialect, whether quoting and delimiter conventions differ between the producer and the consumer of the file. None of these failure modes were available to a printed heading glued to the same physical card as its data, if only because the heading and the punches could not become separated from one another by a software bug. What CSV buys is not the elimination of this risk but its relocation: the header travels with the data through processing steps that preserve the file's structure, and a parser that respects the header can propagate field names correctly through arbitrary schema-preserving downstream processing, though processing that discards or corrupts the header can destroy this association as easily as any other data.

It is also important not to overstate what CSV's header accomplishes even when everything works correctly. Ordinary CSV supplies field names and very little else. It does not say that quantity must be a nonnegative integer, that price is a decimal monetary value denominated in a particular currency, that `town_code` refers to an entry in some authoritative list of municipalities, or that `salesperson_code` is a key into an employee table maintained elsewhere. It specifies no units, no missing-value convention, no validation rule, and no constraint on which categories are permissible values of a given field. All of this remains external to the file, residing in documentation, in the software that produces or consumes it, in institutional convention, or in a separately maintained data dictionary. Of the three admissibility gates defined above, CSV's header addresses none directly; it makes field names machine-readable, which is a precondition for a program to apply type or context checks by name, but it does not itself supply those checks.

The difference between the punched card and CSV is consequently one of degree and location rather than an absolute opposition between an opaque medium and a self-describing one:

Punched card : machine-readable values + human-readable external schema,

CSV with header : machine-readable values+machine-readable field names+external type and warrant semantics.

The punched card's schema is materially superimposed, in the sense that the printed headings sit visibly on the same physical object as the holes, yet computationally external, since the sensing mechanism cannot read print. A human operator perceives both the label Quantity printed above a region of the card and the holes punched within it; the tabulating machine perceives only the holes. CSV closes part of this gap: a parser that respects the header reads both the string quantity and the string 12 as data belonging to the same computational object, and can preserve their association through processing that is designed to preserve it.

This makes the punched card strongly and rigidly positional in a way well-behaved CSV processing need not be. On the card, meaning depends on the convention that a specific physical coordinate plays a specific role; moving a field requires coordinated changes to the card's printed layout, the keypunching practice, the plugboard wiring, and the downstream reporting procedures, all of which must be updated in concert or the record becomes silently misinterpreted. In this narrow but real sense, the card resembles a headerless fixed-width record accompanied by a separately maintained layout specification:

0114SM7012004750030

with the specification stating, elsewhere, that certain columns are the town field, certain others the salesperson field, and so on. Remove that specification and the digits remain intact while their meaning becomes effectively unrecoverable. Printing the field names directly on the card helps a human operator, but because the sensing mechanism cannot read print, those printed names never enter the computational pathway as usable metadata; they exist for people, not for machines.

The resulting asymmetry is worth stating plainly. A human inspecting a punched card can perceive both the schema, via the printed headings, and the instance, via the pattern of holes; the human occupies a privileged vantage point relative to the machine, which perceives only the instance together with whatever schema has been embodied elsewhere in its wiring. With CSV, both the header and the rows enter machine memory together, so a computational system built to respect the header can manipulate, to a limited but genuine extent, not only the values but a partial description of what those values are. This is not a difference of degree in raw computing power; it is a difference in which layer of the overall interpretive chain has been brought inside the artifact that the machine directly reads.

4.1 Further Migration: Declared Types and Explicit Schema

The same progression continues past CSV, and a brief treatment of its further stages clarifies what remains constant across the whole sequence. A relational database table declares, at the point of table creation, not only column names but column types: a quantity column may be declared as an integer, a price column as a fixed-point decimal, and a foreign key constraint may require every value in a `salesperson_id` column to correspond to an existing row in an `employees` table. This is a further internalization of convention: the mechanical substrate, in the form of the database engine's type-checking and constraint-enforcement logic, now refuses certain operations outright rather than silently permitting them and relying on external institutional discipline to prevent their misuse. Attempting to insert a noninteger value into an integer-typed column, or a nonexistent salesperson identifier into a properly constrained foreign key column, is now mechanically rejected rather than merely semantically inadvisable. In terms of the three-gate criterion above, a well-constrained relational schema begins to enforce the second gate, type licensing, in addition to the first.

A schema expressed in a format such as JSON Schema pushes this further still, allowing a data producer to declare not only types but ranges, enumerations of permissible categorical values, required fields, and nested structural relationships, all in a machine-readable specification that can itself be validated against by software before any application logic ever touches the data. Standard JSON Schema can constrain representation, membership, range, and structure, and does a genuinely useful amount of type-gate enforcement as a result. It does not, however, ordinarily specify the complete set of semantically warranted operations over a field; declaring that a price field must be a non-negative number does nothing to prevent an application from averaging prices denominated in different currencies. Additional application logic, a richer domain ontology, or a purpose-built validator would be required to express constraints of that kind, and such tools exist and are used

in some domains, which is to say that the third gate is not in principle unmechanizable, only that it is not addressed by structural schema formats as ordinarily deployed.

It would be a mistake to read this progression as approaching some final state in which the admissibility gap closes entirely, but it would equally be a mistake to say, as an earlier draft of this argument did, that the gap remains undiminished throughout. It does not remain undiminished; it is repartitioned. The punched card enforces only executability, and only through the plugboard rather than the card itself. CSV adds portable field names but essentially no type enforcement of its own. A well-constrained relational schema or a JSON Schema document enforces meaningful portions of type licensing, catching a real and non-trivial class of errors, category mismatches between fundamentally incompatible representations, before they propagate. What none of these formats close, as ordinarily deployed, is the third gate: contextual warrant, the question of whether a mechanically possible and even type-correct operation actually means what its result will be taken to mean. That gap has not moved because it was never a property of the format; it is a property of the relationship between a calculation and the purpose it is meant to serve, and no schema language can fully anticipate every purpose a future user might have for a field it never designed with that purpose in mind.

5 Discussion: Distributed Cognition Across the Sequence

The sequence traced in this essay, from a herder's pebble bowl through tally marks and numerals to the punched card, the CSV file, and the explicitly typed database schema, is not a story of information technology gradually acquiring meaning where previously it had none. Meaning, in the relevant sense, was fully present at every stage: the pebble meant something quite precise relative to the herder's practice, and the punched card's hole meant something quite precise relative to the surrounding apparatus of forms, wiring, and personnel. What changes across the sequence is the location of the convention that supplies that meaning, and, correspondingly, how much of that convention has migrated into a form the machine itself can act upon, as opposed to remaining embodied in practice, printed on a form the machine cannot sense, or held only in the trained judgment of the people operating the system.

This reframes what should and should not be found surprising about the fact, noted at several points above, that a system of coordinated but individually limited components can appear more competent, taken as a whole, than any of its parts. The herder need not compute a cardinality to detect a loss; the pebbles do that work by structural correspondence. The clerk who fills out an order form need not know at what card coordinate a quantity will be punched; the keypunch operator's trained habit and the card's printed layout do that work. The tabulator's wiring need not know what freight means; it needs only to route the correct columns to the correct counters, a fact established once, externally, by an engineer consulting the application's layout specification. The CSV parser need not know that a quantity must be nonnegative; it needs only to recover the string quantity attached to a value, leaving the question of what values are permissible to whatever validation logic, if any, a downstream application chooses to apply. At no point in this chain does any single component, human or mechanical, traverse the entire path from a raw business event to a validated, meaningful aggregate. Competence belongs to the coordinated arrangement of components across the whole chain, not to any one link in it, and this remains true whether the chain in question is a pebble bowl or a modern, professionally administered relational database.

The three-gate admissibility criterion proposed above gives this observation a precise, if modest, formal handle. A system's apparent generality, in the domain-general sense that period marketing materials for tabulating equipment liked to claim and that contemporary claims about data

infrastructure sometimes echo in updated vocabulary, is properly understood as the portability of a small algebra of formal operations, counting, sorting, grouping, summing, joining, across an open-ended range of domains, purchased by externalizing the domain-specific types and contextual warrants that would be required to guarantee that every executable operation is also a meaningful one. No historical stage in the sequence traced here, not the punched card, not CSV, not a fully typed relational schema, closes all three gates at once. Each successive stage absorbs a portion of the convention that used to live entirely outside the artifact into a form the artifact itself, or the software built to read it, can enforce, while leaving a residue, sometimes large and sometimes small but never zero at the level of contextual warrant, dependent on documentation, institutional practice, and trained human judgment exercised outside the machine’s reach.

6 Conclusion

Return, finally, to the gate through which the sheep pass. A pebble deposited in a bowl and a hole punched in a card are, at the level of raw physical description, extremely similar events: a discrete, binary change of state occasioned by a discrete event in the world. The distance between them is not a distance in discreteness, which both systems already possess in full, but a distance in how much surrounding structure, how many simultaneous typed fields, how much of the record’s own description, has been folded into the material artifact rather than left to reside in an accompanying practice, a printed form, a wired board, or a trained mind. The punched card is not more digital than the pebble bowl; it is a far richer descendant of the identical basic maneuver, in which an event in the world licenses a discrete material change whose later interpretation depends upon a convention that the artifact itself only partially preserves. CSV, and the typed schemas that followed it, continue this same migration, each stage bringing a little more of that convention inside the machine-readable channel, absorbing an increasing share of what is executable and what is type-licensed. The deepest layer, contextual warrant, the question of whether a particular operation upon a particular field means what its result will be taken to mean, is not absorbed by any stage considered here. It is repartitioned, again and again, between substrate, artifact, machinery, documentation, and trained judgment, and representational history, on the evidence of this sequence, is the history of that repartitioning rather than a march toward an artifact that finally explains itself.

References

- [1] Computing History. “Introduction of 80-column IBM punch card format.” <https://www.computinghistory.org.uk/det/6168/Introduction%20of%2080-columns%20card%20format>
- [2] National Museum of American History. “IBM Manual Card Punch.” Smithsonian Institution. https://americanhistory.si.edu/collections/object/nmah_694437
- [3] Norman, Jeremy M. “IBM Adopts the Eighty-Column Punched Card, Standard for the Next Fifty Years.” History of Information. <https://www.historyofinformation.com/detail.php?id=596>
- [4] Davey, Tristan. “80 Columns.” Punch Card Archive. <https://punchcards.tristandavey.com/format/80/>
- [5] Shafranovich, Y. “Common Format and MIME Type for Comma-Separated Values (CSV) Files.” RFC 4180, IETF, October 2005.

- [6] Codd, E. F. “A Relational Model of Data for Large Shared Data Banks.” Communications of the ACM 13, no. 6 (1970): 377–387.
- [7] JSON Schema Organization. “JSON Schema Specification.” <https://json-schema.org/specification>